REP-4: Returns and Refunds Report

Instructions for Sean (Part 3, returns and rewards)

Prepared 2026-11-04 from the TA's Checkpoint 3 feedback and the current code. Size M. Owner: Sean.

Item	Details
Your report	A **new** manager-only report, **Returns and refunds**, at **/staff/reports/returns**. Today the store has a returns queue (to process requests) but no report that analyzes returns.
What you add	Return rate by product and category, the most returned products, refund totals as a share of sales, reasons grouped, counts by status, time from request to refund, a backlog of old requests, the refund trend against the previous period, and a 'What this means' panel with actions.
Files you create	`db/reports/returns_analysis.sql`, `db/reports/test_returns_analysis.sql`, `lib/returnsMetrics.js`, `lib/returnsPeriods.js`, `server/reports/returns.js`, `src/app/pages/reports/ReturnsReportPage.jsx` (with a small folder of local components), tests under `tests/`.
Files you change	`src/app/routes.jsx` (one line for the new page). Optional part B: `src/app/pages/returns/RequestReturnPage.jsx` (reason dropdown, section 5).
Files you must leave alone	`lib/returns.js` (it already exists and the checkout and returns endpoints use it; this is why your new file is called `returnsMetrics.js`), `db/core/schema.sql` (frozen), `db/core/views.sql`, `package.json` (REP-7 installs the chart library).
Depends on	Nothing to start. REP-0 (demo data) gives 8 to 10 refunded returns; the backlog and denial panels also need a few requested, approved and denied returns, so add your own rolled-back inserts or ask the lead to extend REP-0 (question 1 at the end).
Done when	The most returned product on the demo data tops the list and the backlog count matches a query run by hand.

Important. SQL in this guide is a hand-checked starting point. It was written against `schema.sql` and follows the style of the existing views, but could not be run here (no SQL Server in the assistant's workspace). Run it on your local SQL Server Express first. Never run anything against the hosted Azure database yourself: the lead loads new SQL there (INT-1).

1. Why this report is needed

The TA's overall comment on all reports was: *"Your reports should go beyond showing what happened and help management understand why it happened, whether performance is improving or declining, and what actions the business should take. Include more meaningful calculations, comparisons, trends, visuals and flexible filters."*

The TA also asked for a report that analyzes **refunds and returns to find frequently returned products and operational problems**. The project has three analytical reports (sales, inventory, rewards) and none of them covers returns, even though Part 3 already records every return with a reason, a status, a handler, timestamps and refund amounts.

What exists today versus what the TA asked for

TA expectation	Today	What you add
Find frequently returned products	Nothing. The staff queue lists one request at a time.	Most returned products, and return rate (returned units / units sold) by product and by category.
Analyze refunds	Refund amounts exist per return, but nobody can see totals or trends.	Refunded amount, share of sales, average refund, and a trend against the previous period.
Find operational problems	The queue counts requests by status only.	Backlog of old requests, average time from request to refund, denial rate, approved but not yet refunded, repeat returners.
Understand why	The reason is a free-text sentence, never summarized.	Reasons grouped (defective, wrong item, not as described, changed mind, other). Optional dropdown so the grouping is clean.
Trends, comparisons, visuals, filters	None.	Filters, KPI cards with changes, charts, sortable tables.
What to do about it	Management must work it out.	A 'What this means' panel: 'Product X has a 22% return rate: check the listing or the supplier'.

The point: today staff handle returns one by one. After this report management sees *which products and reasons cause returns, how much they cost, and whether the process is slow*. Every number should answer one of those questions.

2. The data you will use and its limits

Table	Columns you need	Used for
PRODUCT_RETURN	Return_id, Transaction_id, Employee_id, Requested_at, Status ('requested', 'approved', 'denied', 'refunded'), Reason, Refund_total, Refunded_at	One row per return request.
PRODUCT_RETURN_ITEM	Return_id, Product_id, Quantity, Refund_amount	What was returned and the refund per product.
SALES_TRANSACTION	Transaction_id, Customer_id, Transaction_date	Days between purchase and return, customer.
TRANSACTION_ITEM	Transaction_id, Product_id, Quantity, Line_total	Units purchased on the original line.
PRODUCT, CATEGORY	Name, SKU, Product_condition (New, Used, Refurbished), Category_id	Grouping.
VW_SALES_REPORT (existing view)	Sale_date, Units, Total_collected	Denominators: units sold and money collected.
EMPLOYEE (optional)	Employee_id, FirstName	Who processed a return (counts only).

Rule. Every query and every join must filter **Marked_for_deletion = 0**, including the lookup tables. The application login cannot DELETE rows; the project uses soft delete everywhere. Note that the returns code counts soft-deleted rows when it checks quantities (on purpose); the report must not, so deleted rows never inflate a total.

How returns work today (so your numbers are right)

- A return goes **requested**, then **approved** or **denied** by staff, then **refunded**. Marking it refunded sets `Refunded_at`; the database trigger then puts the units back in stock and takes the points back (`Reason = 'Return refund'`). The application never touches stock or points.
- The refund for a line is the line's price after discounts **plus its share of the sale's tax** (see `lib/returns.js`). So the cash refunded is a little higher than the 'Refunds' KPI of the Sales report, which leaves tax out. Say so on the page.
- A customer can return within **30 days** of the sale. Sales from the last 30 days can still produce returns, so recent return rates are understated. Say so on the page.
- **There is no 'approved at' time.** Only `Requested_at` and `Refunded_at` exist. Time to refund can only be measured for refunded returns; approved or requested returns are measured by how long they have been waiting.
- **Reason is free text** (up to 200 characters). Group by the cleaned text; see the dropdown in part B.
- The handler (`Employee_id`) is empty while a return is requested.
- The refund trigger puts every returned unit back in stock, even a defective one. That is outside this task, but worth telling the lead (question 3).

Limits you must respect and print on the page

- **Returned units** = units on returns that are not denied (requested, approved, refunded). **Refunded cash** = refund amounts of returns with status refunded only. Print both definitions.
- **Request date decides the period.** The report filters returns by the day they were requested (what happened in the period), and compares them with units sold in the same period.
- **The seed has no returns at all.** Every panel needs an empty state ('no returns in this period').
- **Small numbers lie.** A product that sold 2 units and had 1 returned has a 50% rate. Rank rates only for products with at least 5 units sold (parameter `minSold`), and list the rest as 'low volume'.
- **Privacy:** the page is managers only. Customer names may appear in the repeat-returners list; never passwords or emails.

3. Parameters (filters)

The new endpoint is GET /api/reports/returns (manager only). It works with no parameters (last 30 days). All parameters are optional, validated on the server, echoed back in the response, and kept in the page URL so a filtered view can be shared.

Parameter	Values	Default	Effect and validation
from, to	YYYY-MM-DD	last 30 days	Returns requested in the range. Real calendar date check (copy `realDate` from `server/reports/sales.js`), start not after end, range at most 400 days (HTTP 400).
compare	previous, lastyear, none	previous	previous = equal-length period immediately before `from`. lastyear = same dates one year earlier. none = no change figures.
category	A Category_id (integer)	all	Only returned items of that category.

Parameter	Values	Default	Effect and validation
product	A Product_id (integer)	all	Only that product.
status	requested, approved, denied, refunded	all	Only returns in that status. The backlog and status counts always use all statuses.
reason	defective, wrong_item, not_as_described, changed_mind, other	all	Only returns whose cleaned reason falls in that group (section 4). Anything else: 400. Apply after grouping, in JavaScript.
condition	New, Used, Refurbished	all	Only products of that condition.
groupBy	day, week, month	day	Buckets of the trend chart. Weeks start on Monday. Over 92 days with day, switch to week and return `groupByUsed`.
minSold	1 to 100	5	Minimum units sold before a product gets a return rate.
backlogDays	1 to 30	3	A requested or approved return older than this counts as backlog.
limit	1 to 20	10	Length of the most returned products and repeat returners lists.

Date presets (last 7 days, last 30 days, this quarter, year to date) are page buttons that fill `from` and `to`.

Coordination. REP-7 will later provide a shared `lib/reportPeriods.js`. Until it merges, put your own helper in `lib/returnsPeriods.js` with names `previousPeriod`, `lastYearPeriod`, `buckets`. Terry (REP-2) and Ben (REP-3) do the same in their own files; the small duplication is deliberate so nobody waits for anybody. REP-7 replaces all three.

4. Metrics and formulas

Keep every calculation in pure functions in `lib/returnsMetrics.js` (not in `server/`, where every file becomes a Vercel route). Functions take plain rows and return plain values, so tests need no database.

Metric	Formula	Notes
Returns requested	COUNT(DISTINCT Return_id) requested in the period	All statuses, after filters.
Returned units	SUM(Quantity) on returns not denied	See limits above.
Units sold	SUM(Units) of VW_SALES_REPORT in the period (or per product and category from VW_UNITS_SOLD_DAILY)	The denominator.
Return rate	Returned units / units sold x 100	Overall, per category and per product. Null when units sold is 0. Compared in points.
Refunded cash	SUM(Refund_amount) on refunded returns	Includes the tax share. Attributed to the request date.
Refund share of sales	Refunded cash / Total_collected x 100	Total_collected comes from VW_SALES_REPORT, which also includes tax, so both sides match.
Average refund	Refunded cash / refunded returns	
Time to refund	Refunded_at - Requested_at, in hours	Refunded returns only. Show the average and the median; show days when over 48 hours.

Metric	Formula	Notes
Days open	Today - Requested_at, for requested or approved returns	Used for the backlog.
Backlog	Returns with status requested or approved and days open over backlogDays	Also report the oldest, and requested and approved separately (approved but not refunded means the refund step is stuck).
Backlog age buckets	0 to 1, 2 to 3, 4 to 7, 8 or more days	For the aging chart.
Denial rate	Denied returns / returns decided (approved, denied, refunded) x 100	Requested returns are not decided yet and are excluded.
Days after purchase	Request date - sale date	Buckets 0 to 7, 8 to 14, 15 to 30. Shows how quickly customers notice problems.
Reason group	Cleaned reason text mapped to a group	Rules below.
Returns per customer	Returns in the period per customer	Repeat returner = 3 or more returns in the period (constant REPEAT_RETURNER).
Change (percent)	(current - previous) / previous x 100	Null when previous is 0 (show 'new'). Rates change in points.

Cleaning and grouping the reason

- Trim spaces, lower-case, and take the text before the first colon if there is one (the dropdown in part B writes 'Defective or damaged' or 'Other: free text').
- Map to a group: **defective** (contains 'defect', 'damag', 'broken', 'not work', 'scratch'), **wrong_item** ('wrong', 'incorrect', 'different'), **not_as_described** ('described', 'expect', 'misleading', 'fake'), **changed_mind** ('changed', 'no longer', 'mind', 'do not want'), otherwise **other**. Keep the keyword lists as constants at the top of the library.
- Show 'other' with the 5 most common raw texts, so management can see what hides inside it.

5. Build procedure

Step 1. Create the views (db/reports/returns_analysis.sql)

A view cannot take parameters, so the views hold the joins and the endpoint adds the filters. Use `CREATE OR ALTER VIEW` so the file can be run again.

```
-- VW_RETURNS_ANALYSIS: one row per returned item (not flagged), with the original sale line
CREATE OR ALTER VIEW VW_RETURNS_ANALYSIS AS
SELECT r.Return_id, r.Transaction_id, s.Customer_id, r.Employee_id,
       r.Requested_at, CAST(r.Requested_at AS DATE) AS Request_date,
       CAST(s.Transaction_date AS DATE) AS Sale_date,
       DATEDIFF(DAY, CAST(s.Transaction_date AS DATE), CAST(r.Requested_at AS DATE))
                                                              AS Days_after_purchase,
       r.Status, r.Reason, r.Refunded_at,
       CASE WHEN r.Status = 'refunded'
            THEN DATEDIFF(HOUR, r.Requested_at, r.Refunded_at) END AS Hours_to_refund,
       CASE WHEN r.Status IN ('requested', 'approved')
            THEN DATEDIFF(DAY, CAST(r.Requested_at AS DATE), CAST(SYSDATETIME() AS DATE))
            END AS Days_open,
       ri.Product_id, p.SKU, p.Name AS Product_name, p.Product_condition,
       c.Category_id, c.Name AS Category_name,
       ri.Quantity, ri.Refund_amount,
       ti.Quantity AS Units_purchased,
```

```
       CAST(ti.Line_total AS DECIMAL(12,2)) AS Line_total
FROM   PRODUCT_RETURN r
JOIN   PRODUCT_RETURN_ITEM ri ON ri.Return_id = r.Return_id AND ri.Marked_for_deletion = 0
JOIN   SALES_TRANSACTION s ON s.Transaction_id = r.Transaction_id AND s.Marked_for_deletion = 0
JOIN   TRANSACTION_ITEM ti ON ti.Transaction_id = r.Transaction_id
                        AND ti.Product_id = ri.Product_id AND ti.Marked_for_deletion = 0
JOIN   PRODUCT p  ON p.Product_id  = ri.Product_id AND p.Marked_for_deletion = 0
JOIN   CATEGORY c ON c.Category_id = p.Category_id  AND c.Marked_for_deletion = 0
WHERE  r.Marked_for_deletion = 0;
GO

-- VW_UNITS_SOLD_DAILY: units and revenue of completed sales per day and product
CREATE OR ALTER VIEW VW_UNITS_SOLD_DAILY AS
SELECT CAST(s.Transaction_date AS DATE) AS Sale_date,
       ti.Product_id, p.Category_id, p.Product_condition,
       SUM(ti.Quantity) AS Units,
       CAST(SUM(ti.Line_total) AS DECIMAL(12,2)) AS Revenue
FROM   SALES_TRANSACTION s
JOIN   TRANSACTION_ITEM ti ON ti.Transaction_id = s.Transaction_id AND ti.Marked_for_deletion = 0
JOIN   PRODUCT p ON p.Product_id = ti.Product_id AND p.Marked_for_deletion = 0
WHERE  s.Status = 'completed' AND s.Marked_for_deletion = 0
GROUP BY CAST(s.Transaction_date AS DATE), ti.Product_id, p.Category_id, p.Product_condition;
GO
```

Check: run the file twice (it must not fail the second time). Pick one refunded return and compare its row in `VW_RETURNS_ANALYSIS` with the tables by hand (quantity, refund amount, days after purchase, hours to refund). Then check that `SUM(Units)` of the second view over a day equals `Units` of `VW_SALES_REPORT` for that day.

> **Why it cannot double count.** The first view joins the sale line with a plain JOIN. This is safe because `TRANSACTION_ITEM` has one row per sale and product (its primary key), so a returned item matches exactly one line.

Step 2. Write a rolled-back test script (db/reports/test_returns_analysis.sql)

Follow the pattern of `db/core/test_guards.sql` and `verify.sql`: open a transaction, insert known rows, assert the views, then ROLLBACK so nothing is left behind. Cover at least:

- A sale of 2 units with 1 unit returned: Quantity 1, Units_purchased 2, Days_after_purchase correct.
- A refunded return: Hours_to_refund correct, Days_open NULL. A requested return 5 days old: Days_open = 5, Hours_to_refund NULL.
- A denied return: present in the view with Status 'denied' (the endpoint decides what counts).
- A flagged return, a flagged return item and a flagged product: not in the view.
- Units sold per day in the second view agree with VW_SALES_REPORT.

The return guard triggers reject invalid rows (for example more units than were bought). Build your test data the way `test_guards.sql` does, with valid amounts, so the guards accept them.

Step 3. Write the pure metric and period modules

Function	Input	Output
previousPeriod, lastYearPeriod, buckets	Dates and 'day', 'week' or 'month'	Equal-length previous period, same dates last year, ordered buckets without gaps.
reasonGroup(text)	Raw reason text	'defective', 'wrong_item', 'not_as_described', 'changed_mind' or 'other'.
summarize(rows, soldUnits, collected)	View rows, units sold, money collected	{ returns, returnedUnits, returnRatePct, refundedCash, refundSharePct, avgRefund, avgHoursToRefund, medianHoursToRefund, denialRatePct }.
compareKpis(current, previous)	Two summaries	Each key as { value, previous, change, changeKind } with kind 'percent', 'points' or 'new'.
rankProducts(rows, soldByProduct, minSold, limit)	Rows and sold units	{ mostReturned, highestRate, lowVolume }.
rateByGroup(rows, soldByGroup)	Rows and sold units per category or condition	Return rate per group, with the store average.
backlog(rows, backlogDays)	View rows	{ requested, approved, total, oldestDays, buckets }.
reasons(rows)	View rows	Counts per reason group and the top raw texts inside 'other'.
trendSeries(rows, buckets)	Rows and buckets	Arrays aligned to the buckets: returns, refundedCash.
repeatReturners(rows, limit)	Rows	Customers with 3 or more returns, with count and refunded cash.
buildReturnsInsights(...)	The summaries above	Up to 5 { id, severity, text, action }. See section 7.

Use integer cents when summing money so totals never drift by a cent.

Step 4. Write the endpoint (server/reports/returns.js)

1. Start with `requireAuth(req, ['manager'])` and validate the parameters of section 3 (`HttpError(400, ...)` for bad values; `validate.integer` for numbers).
2. Work out the comparison dates with `returnsPeriods`.
3. Query `VW_RETURNS_ANALYSIS` for the period (and the comparison period) with named parameters only. Add category, product and condition filters in SQL only when present.
4. Query the units sold in the period per product and category from `VW_UNITS_SOLD_DAILY`, and the units and money collected from `VW_SALES_REPORT`.
5. Query the backlog separately **without** the date filter: every requested or approved return, whatever its age (the backlog does not depend on the chosen period).

6. Compute everything with the library, apply the status and reason filters in JavaScript, and return: **filters, period, comparePeriod, groupByUsed, kpis, trend, mostReturned, highestRate, lowVolume, byCategory, byCondition, reasons, daysAfterPurchase, statusCounts, backlog, repeatReturners, insights**.
7. Run **npm run routes** so `lib/routes.generated.js` includes the new endpoint, and commit it. `npm test` fails if the generated list is stale.

> **Keep.** The staff queue endpoints (`server/returns/`) and the 93 tests in `tests/returns.test.js` must keep passing. You are only adding a new file next to them.

Step 5. Build the page (ReturnsReportPage.jsx) and register it

1. Add one route line to `src/app/routes.jsx` (path `/staff/reports/returns`, manager access, nav label 'Returns report', summary 'Return rates, refunds and processing delays.'). REP-7 also edits that file; keep your change to that one line and tell the REP-7 owner.
2. Put new small components in `src/app/pages/reports/returns/` (filter bar, KPI card, product table, backlog panel, insights). When REP-7 merges you replace them with the shared ones.
3. Layout, top to bottom: header, filter bar, KPI cards, 'What this means' panel, trend and most returned products, reasons and category charts, backlog table with a link to the returns queue, repeat returners. The wireframe on the last page shows it.
4. Read the filters from the URL query string and write them back when they change.
5. Every KPI card shows its value, the change in words and colour, and the previous value in small text. For this report **lower is better** for return rate, refunded cash, time to refund and backlog, so colour by meaning, not by sign.
6. The backlog panel lists each old return (id, product, days open, status) with a link to the existing queue page so staff can act on it.
7. Charts: until REP-7 adds a chart library, draw bars with plain `div` elements and inline styles (allowed by the site's security policy) and the trend as an inline SVG line.
8. Empty states: no returns in the period; no data in the comparison period; nothing in the backlog ('all requests are up to date').

Part B (optional but recommended): a reason dropdown on the return form

The report groups free text, which works but is fuzzy. A dropdown in `RequestReturnPage.jsx` makes the grouping exact without any database or server change:

- Options: Defective or damaged, Wrong item received, Not as described, Changed my mind, Other.
- The form sends the same `reason` text as today: the option name alone, or 'Other: ' followed by the customer's words (still limited to 200 characters). The server and the 93 tests do not change.
- Old free-text returns still work because the report cleans text and maps keywords.
- Do this only after the report works; it is a few lines of front-end code.

Step 6. Tests

Test file	What it checks	How
tests/lib/returnsMetrics.test.js	Return rate with zero sales, denied returns excluded from returned units, refunded cash only from refunded, median time, backlog buckets, denial rate excluding requested, reason grouping (keywords, colon prefix, blank), min units rule, repeat returners, insight rules and ordering.	Plain fixtures, no database.
tests/lib/returnsPeriods.test.js	Previous period, last year (29 February), buckets without gaps, over 92 days switches to week.	Plain fixtures.
tests/returnsReport.test.js	Bad dates, category, product, status, reason, condition, minSold, backlogDays or limit give 400; valid filters reach the query as named parameters; the backlog query has no date filter; support 403; no login 401; SQL text contains Marked_for_deletion = 0.	Use `tests/helpers_manager.js` with `jest.mock('../lib/db', ...)` as in `tests/catalog.test.js`.

Do not use `helpers_backoffice.js` for filter tests: its request builder does not support query parameters.

6. Visual KPIs and chart ideas

The reader first sees the cards, then the meaning panel, then the evidence in charts. Each item says what question it answers; cut anything that does not answer one.

6.1 KPI cards

Card	Value	Change shown	Question it answers
Returns requested	Count in the period	Percent change	Are more customers sending things back?
Return rate	Returned units / units sold	Points up or down (lower is better)	Is a bigger share of what we sell coming back?
Refunded cash	Sum of refunded amounts, and its share of sales	Percent change	What do returns cost us?
Time to refund	Average and median hours from request to refund	Percent change (lower is better)	Is the process fast enough?
Open backlog	Requested or approved returns older than the limit	Oldest age in days	What needs attention today?
Denial rate	Denied / decided returns	Points up or down	Is the policy clear, or is it being abused?

6.2 Charts

Chart	Type	What it shows
Returns and refunds trend	Line, refunded cash per bucket, with the previous period dashed; count as a second series or bars	Whether returns are improving or worsening (the TA's wording).
Most returned products	Horizontal bars of returned units, with the return rate in the label; low-volume products greyed	Frequently returned products: the TA's main question.
Return reasons	Donut or horizontal bars by reason group	Why customers return things. A big 'defective' slice points to suppliers or quality.
Return rate by category	Bars with a dashed store-average line	Which category returns most.
Return rate by condition	Three bars: New, Used, Refurbished	Whether used or refurbished stock causes more returns.
Days after purchase	Histogram: 0 to 7, 8 to 14, 15 to 30 days	How fast customers notice a problem; a pile at day 28 to 30 suggests customers decide late.
Backlog aging	Bars: 0 to 1, 2 to 3, 4 to 7, 8 or more days, older buckets red	Operational problem at a glance.
Status funnel	Bars: requested, approved, refunded, denied	Where requests end up.
Time to refund	Histogram of hours	Typical and worst processing time. Extra if time allows.

Minimum for 'done': the trend, most returned products, reasons and the backlog list. The others are extra credit.

7. 'What this means' panel: the insight rules

These are rule-based sentences, not guesses. Each rule has a condition, a sentence built from real numbers and a recommended action. Show at most five, most severe first. Put the rules in `buildReturnsInsights` so they can be tested. Skip comparison rules when compare is none or the previous period has no returns.

Id	Severity	Condition	Sentence and action
backlog	Danger	Backlog above 0	"N return requests have waited more than 3 days (oldest: 9 days)." Action: open the returns queue (link).
stuck	Danger	Approved returns older than the limit	"N approved returns have not been refunded for more than 3 days." Action: finish the refund step.
productRate	Warning	A product with at least minSold units sold and a return rate of 15% or more	"Product X has a 22% return rate (4 of 18 units)." Action: check the listing, the condition or the supplier.
reason	Warning	One reason group above 40% of returns (and at least 5 returns)	"Defective or damaged is 45% of returns." Action: inspect stock from the suppliers of the top products (open the inventory report).
slow	Warning	Median time to refund above 72 hours	"The median refund takes 4 days." Action: add handling capacity or review the steps.

Id	Severity	Condition	Sentence and action
spike	Warning	Refunded cash up 25% or more on the previous period	"Refunds rose 38% to $1,240." Name the product or category with the largest increase. Action: investigate that product.
denial	Warning	Denial rate of 30% or more (at least 5 decisions)	"31% of decided returns were denied." Action: review the denial reasons and make the policy clearer to customers.
repeat	Info	Customers with 3 or more returns	"N customers returned 3 or more times (together $X refunded)." Action: review these accounts.
condition	Info	Used or Refurbished rate at least twice the New rate	"Used items are returned 2.4 times as often as new ones." Action: test used stock before selling.
category	Info	A category at least 1.5 times the store rate	"Accessories return 1.8 times more than average." Action: review that category's suppliers.
allgood	Good	No danger or warning	"Returns are under control: the backlog is clear and the return rate is stable."

8. Traps to avoid

- **Counting denied returns as returned.** Returned units exclude denied; refunded cash counts refunded only.
- **Quoted amounts.** `Refund_amount` on a requested or approved return is only the amount that would be refunded. Never add it to refunded cash.
- **Soft delete.** A forgotten `Marked_for_deletion = 0` on a joined table brings back deleted returns. Test it.
- **Date columns.** `Requested_at` is a DATETIME2. Filter the view's `Request_date` column (a DATE) with BETWEEN, as `sales.js` does.
- **Backlog depends on today, not on the period.** Do not apply the date filter to it.
- **Tiny samples.** Use the minimum units rule; never rank a 2-unit product by percent.
- **Immature recent data.** Return rates for the last 30 days are understated because customers still have time to return. Print the note.
- **Tax in refunds.** Refunded cash includes tax; compare it only with money collected (also with tax), never with subtotal.
- **Percent of zero.** A previous value of 0 gives 'new', never infinity or NaN. Rates change in points.
- **Colour alone.** Pair red or green with words ('worse', 'better').
- **Server files become routes.** Put helper code in `lib/`, not in `server/`, and do not edit `lib/returns.js`.
- **Time zone.** 'Today' is the server date (UTC on Azure). Acceptable; note it in the PR.

9. Definition of done

[ ]	Both views are in `db/reports/returns_analysis.sql`, run twice without errors on a local database, and every join filters Marked_for_deletion = 0.
[ ]	The rolled-back test script passes and leaves no rows behind.
[ ]	One refunded return checked by hand: quantity, refund amount, days after purchase and hours to refund match what the report shows.
[ ]	Pure functions in `lib/` have tests for every formula, including zero sales, denied returns and the minimum units rule.
[ ]	The endpoint is manager-only and tested (200, 403, 401, 400); every parameter is validated.
[ ]	`npm run routes` was run and the generated file committed; `npm test` passes, including the 93 existing returns tests.
[ ]	The most returned product on the demo data tops the list, and the backlog count matches a query you ran by hand.
[ ]	The page has: filter bar, six KPI cards with changes, meaning panel with 3 to 5 insights, at least the trend, most returned products and reasons, the backlog list with a link to the queue, empty states.
[ ]	The page states its limits: returned units and refunded cash definitions, tax in refunds, immature recent data.
[ ]	The new route appears in the manager menu; staff without the manager role cannot open it.
[ ]	The PR description lists the parameters, the new files and the manual checks you did.

Questions to ask the lead before you start: (1) Can REP-0 also create a few requested, approved and denied returns, some more than 3 days old? Without them the backlog and denial panels cannot be demonstrated. (2) Do you want the reason dropdown (part B)? Suggested: yes. (3) The refund trigger restocks every returned unit, even defective ones; should the report or the process say anything about that? Suggested: only mention it in the demo notes.

Wireframe: how the Returns and refunds report page could look

Reading order: filters, six KPI cards with changes, the 'What this means' panel (right), charts (left), then the backlog table (with a link to the queue), the most returned products table and the repeat returners. The numbers are examples only.

Lower is better here: return rate, refunded cash, time to refund and backlog all improve when they fall, so colour follows meaning and the words ('worse', 'up', 'down') are always printed. Until REP-7 installs a chart library, draw bars with plain div elements and the trend as an inline SVG line.